

# Introduction to B-Tree & B+ Tree Data Structures

> **Definition:** A **B-Tree** is a self-balancing multi-way search tree of order  $m$  where each node can contain multiple keys and more than two children, designed specifically to optimize **disk read/write block accesses** in database indexing and file systems.

## 1. Detailed Technical Explanation

### 1. Properties of a B-Tree of Order $m$ :

1. Every node has at most  $m$  children.
2. Every leaf node contains pointers to the next leaf node.

## 2. B-Tree vs B+ Tree Comparison

B+ TREE:

### Detailed Comparison Table:

| Feature        | B-Tree                                 | B+ Tree                                 |
|----------------|----------------------------------------|-----------------------------------------|
| Leaf Nodes     | Not linked sequentially                | Linked sequentially                     |
| Internal Nodes | Store keys and pointers to child nodes | Store only keys, pointers to leaf nodes |
| Search Path    | May vary for same key                  | Always same for same key                |
| Range Queries  | Less efficient                         | More efficient                          |
| Insertion      | May cause rebalancing                  | Less frequent                           |
| Deletion       | May cause rebalancing                  | Less frequent                           |

---

### 3. Core Concepts & Memory Keywords

- **Order m:** Maximum number of children a node can have.

- **F**

---

### 4. Must-Write Points for Exams

- B-Trees and B+ Trees reduce disk I/O by fitting large numbers of keys inside a single disk page block (e.g., 4KB or 8KB).

- **B+**

---

### 5. Quick Recall Flow

---

Multi-V